Inteligență Artificială

Căutări locale și on-line

Slides: Andrei Olaru, Adina Florea

- Căutare locală
- Căutare on-line

- o căutare locală este o strategie de căutare bazată pe mutarea dintr-o stare curentă într-o stare vecină.
- sunt utile atunci când
  - spațiul de căutare este mare și/sau nu avem suficient spațiu pentru a păstra în memorie un număr mare de stări
  - nu ne interesează (optimalitatea) drumul(ui) până la soluție
  - ne interesează starea finală în care ajungem
- e.g., util în probleme de tip CSP

- alege întotdeauna o stare vecină pentru care o funcție de evaluare este mai bună.

Hill Climbing(s_0, *eval*, *limita*)

1. $s \leftarrow s_0$
2. **cât timp** *iter* $<$ *limita*
3. $\quad$ *iter* $\leftarrow$ *iter* $+ 1$
4. $\quad$ *Succ* $\leftarrow$ *calculeaza_succesori*(s)
5. $\quad$ **dacă** $\exists s_j \in$ *Succ*, s_j stare finală **atunci întoarce** s_j
6. $\quad$ găsește $s' \in$ *Succ* cu *eval*(s') $\geq$ *eval*(s)
7. $\quad$ $s \leftarrow s'$
8. **întoarce** EȘEC

- alege întotdeauna o stare vecină pentru care o funcție de evaluare este mai bună.

Hill Climbing(s_0, *eval*, *limita*)

1. $s \leftarrow s_0$
2. **cât timp** *iter* $<$ *limita*
3. $\quad$ *iter* $\leftarrow$ *iter* $+ 1$
4. $\quad$ *Succ* $\leftarrow$ *calculeaza_succesori*(s)
5. $\quad$ **dacă** $\exists s_j \in$ *Succ*, s_j stare finală **atunci întoarce** s_j
6. $\quad$ alege primul $s' \in$ *Succ* cu *eval*(s') $\geq$ *eval*(s) *first-choice HC*
7. $\quad$ $s \leftarrow s'$
8. **întoarce** EȘEC

- alege întotdeauna o stare vecină pentru care o funcție de evaluare este mai bună.

Hill Climbing(s_0, *eval*, *limita*)

1. $s \leftarrow s_0$
2. **cât timp** *iter* $<$ *limita*
3. $\quad$ *iter* $\leftarrow$ *iter* $+ 1$
4. $\quad$ *Succ* $\leftarrow$ *calculeaza_succesori*(s)
5. $\quad$ **dacă** $\exists s_j \in$ *Succ*, s_j stare finală **atunci întoarce** s_j
6. $\quad$ $s' \leftarrow$ *argmax*$_{s_j \in Succ}$ *eval*(s_j) *steepest ascent HC*
7. $\quad$ $s \leftarrow s'$
8. **întoarce** EȘEC

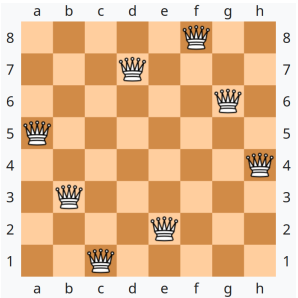
- alege întotdeauna o stare vecină pentru care o funcție de evaluare este mai bună.

Hill Climbing(s_0, *eval*, *limita*)

1. $s \leftarrow s_0$
2. **cât timp** $iter < limita$
3. $iter \leftarrow iter + 1$
4. $Succ \leftarrow calculeaza_succesori(s)$
5. **dacă** $\exists s_j \in Succ$, s_j stare finală **atunci întoarce** s_j
6. alege aleator $s' \in Succ$ cu $eval(s') \geq eval(s)$ *stochastic HC*
7. $s \leftarrow s'$
8. **întoarce** EȘEC

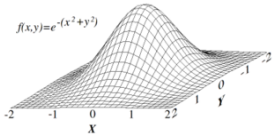
Exemplu: problema celor 8 regine

- pentru soluție, reginele nu trebuie să se atace reciproc
- putem fixa fiecare regină pe o linie
- putem porni de la o stare oarecare
- la o mutare putem muta o regină pe o altă coloană $\Rightarrow$ 56 de vecini pentru fiecare stare. Avem în total $8^8 \approx 16$M stări
- evaluăm o stare ca numărul de perechi de regine care se atacă reciproc

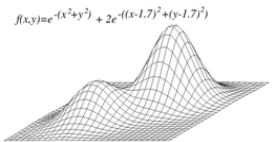


Wikipedia

Hill climbing găsește soluția atunci când funcția *eval* este *convexă* pe spațiul stărilor.



Dacă există mai multe multe puncte de maxim local, este posibil ca HC să găsească un maxim local.



Dacă există platouri, este posibil ca HC să se blocheze.



- Pentru a rezolva o parte din problemele care pot apărea, putem folosi Random restart Hill Climbing

Random Restart Hill Climbing(N, *eval*, *limita*)

1. **pentru** $i = \overline{1, N}$
2. alege o stare s_0
3. *rezultat* $\leftarrow$Hill Climbing(s_0, *eval*, *limita*)
4. **dacă** *rezultat* $\neq$ EȘEC **atunci întoarce** rezultat
5. **întoarce** EȘEC

- Dacă se observă că pentru o problemă HC are probabilitate p de reușită, putem alege $N = \frac{1}{p}$

Căutare locală : Simulated annealing

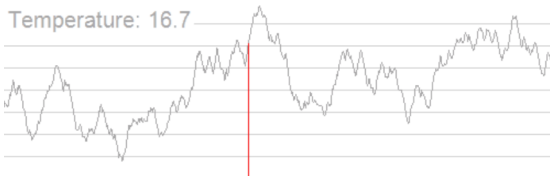
- inspirat de procesul de călire din metalurgie (*annealing*)
- considerăm că stările au o anumită "energie" și căutăm starea de energie minimă globală
- mergem într-o stare vecină mai bună dacă am găsit una, dar putem merge, cu o anumită probabilitate, și într-o stare mai proastă
- probabilitatea de a alege o stare mai proastă scade pe măsură ce algoritmul progresează
- dacă algoritmul este bine proiectat, poate găsi minimul global și atunci când există un număr mare de minime locale

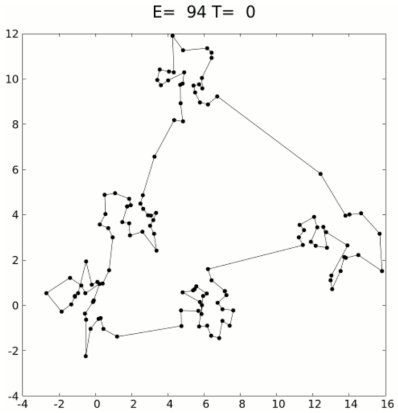
Simulated Annealing(T_{init}, T_{final}, *cooling*, *eval*, *alege_succesor*)

1. $s \leftarrow s_0$; $T \leftarrow T_{init}$
2. **cât timp** $T > T_{final}$
3. $s' \leftarrow alege_succesor(s)$
4. **dacă** $eval(s') < eval(s)$
5. **atunci** $s \leftarrow s'$
6. **altfel** $s \leftarrow s'$ cu probabilitate $e^{-\frac{eval(s')-eval(s)}{T}}$
7. $T \leftarrow cooling \cdot T$

- graful stărilor trebuie să aibă un diametru redus – chiar dacă numărul de stări este foarte mare, trebuie să existe un drum relativ scurt între orice stare și starea finală
  - e.g., pentru problema comis-voiajorului cu 20 de orașe, chiar dacă sunt $2 \cdot 10^{18}$ stări, fiecare stare are 190 de vecini și diametrul grafului este 19.
- poate fi util ca generarea stărilor succesor să producă stări de energie apropiată cu starea curentă, pentru a evita mutarea în stări foarte proaste.
  - e.g. pentru comis-voiajor, succesorii sunt drumuri în care se inversează 2 orașe, distanța dintre orașele inversate scade (statistic) pe măsură ce algoritmul progresează
- algoritmul este vulnerabil la seturi mari de stări cu energie foarte mică față de vecinii lor, dar nu global minimă (spații de stări numite “bazine”).
- se pot folosi reporniri – reveniri la stări de energie mică obținute în trecut, dacă au trecut un număr de pași în stări cu energie semnificativ mai mare decât minima găsită.

Căutare locală : Simulated annealing



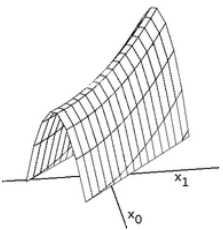


Wikipedia

- la fel ca Beam Search, dar pornește cu B stări generate aleator (în loc să pornească doar cu s_0)
- Stochastic Local Beam Search – alege stochastic cele B stări de păstrat în *beam*, cu o probabilitate proporțională cu calitatea stărilor → ajută la evitarea maximelor locale

În mediile continue, numărul de stări vecine este infinit. Putem utiliza

- discretizarea spațiului, realizând mutări cu un pas fix (sau cu un pas preogresiv mai mic)
  - o problemă suplimentară apare când există "culmi" (*ridges*) nealiniate cu una dintre axe, în care trebuie să ne deplasăm pe mai multe direcții simultan pentru a obține o evaluare mai bună.



- gradient descent / ascent – derivăm local funcția de evaluare pentru a decide direcția de urmat (cu gradientul cel mai pronunțat)

Căutare on-line

- în căutarea “off-line”, agentul are acces la toate stările problemei și poate alege să calculeze succesorii oricărei stări în orice moment

- în căutarea “on-line”, agentul poate alege ca stare următoare doar o stare vecină cu starea curentă – agentul realizează căutarea în timp ce acționează în mediu.
  - NOTĂ: termenul nu are legătură cu activitatea *online* în Internet.

Căutare on-line

- în căutarea “off-line”, agentul are acces la toate stările problemei și poate alege să calculeze succesorii oricărei stări în orice moment

- în căutarea “on-line”, agentul poate alege ca stare următoare doar o stare vecină cu starea curentă – agentul realizează căutarea în timp ce acționează în mediu.
  - NOTĂ: termenul nu are legătură cu activitatea *online* în Internet.

- într-o problemă de căutare online, HC nu poate fi folosit în conjuncție cu Random Restart
- similar, Simmulated Annealing funcționează doar dacă *alege_succesor* alege o stare vecină în mediul fizic al căutării
- BFS nu se poate aplica
- la DFS, revenirea aduce un cost
- la A*, nu putem alege orice nod pentru a fi explorat la următoarea iterație

Căutare on-line : On-line DFS

- variantă de backtracking iterativ unde revenirea se face tot prin acțiuni care trebuie realizate în mediu

Căutare on-line : On-line DFS

On-line DFS(s_0)
global: *Urma*, *De_vizitat*, *Rezultat*

1. $s \leftarrow s_0$, $s_{prev} = \perp$, $a_{prev} = \perp$
2. *Urma*[s] $\leftarrow \emptyset, \forall s$
3. **cât timp** $s \neq$ SUCCES și $s \neq$ EȘEC
4. $\quad a \leftarrow pas(s)$
5. $\quad s \leftarrow aplica_actiune(s, a)$

pas(s) – întoarce acțiunea de realizat din s

1. **dacă** s este stare finală **atunci întoarce** SUCCES
2. **dacă** $s_{prev} \neq \perp$ **atunci**
3. $\quad Rezultat[s_{prev}, a_{prev}] \leftarrow s$
4. $\quad Urma[s] \leftarrow \{s_{prev}\} \cup Urma[s]$
5. **dacă** s este stare nouă
6. $\quad$ **atunci** $De_vizitat[s] \leftarrow actiuni(s)$
7. **dacă** $De_vizitat[s] = \emptyset$ **atunci**
8. $\quad$ **dacă** $Urma[s] = \emptyset$ **atunci întoarce** EȘEC
9. $\quad s_{prev} \leftarrow \perp$
10. $\quad rev = Urma[s].pop()$
11. $\quad$ **întoarce** o acțiune a a.î. $Rezultat[s, a] = rev$
12. $s_{prev} \leftarrow s$
13. **întoarce** $De_vizitat[s].pop()$

Căutare on-line : ADP

- în programarea dinamică (*dynamic programming – DP*), lucrăm prin obținerea soluției la diverse sub-probleme, care pot să nu fie disjuncte
- soluția globală este optimă dacă orice componentă a soluției este soluție optimă pentru o subproblemă
- dar avem nevoie de acces la întreg spațiul de stări al problemei pentru a folosi DP.
- pentru spații de stări foarte mari, este costisitor să parcurgem chiar și o singură dată întregul spațiu

Căutare on-line : ADP

Asynchronous Dynamic Programming – ADP

- dacă am cunoaște h^* pentru fiecare nod, am putea obține drumul optim, prin căutare locală, astfel:

1. **cât timp** nu am ajuns la starea finală
2. **pentru fiecare** fiecare $s_j \in$ *calculeaza_succesori*(s)
3. $f^*(s_j) = c(s, s_j) + h^*(s_j)$ cu $c(a, b)$ costul muchiei (a, b)
4. $s \leftarrow argmin_{s_j} f^*(s_j)$

- putem actualiza asincron estimarea $h[s]$ pentru fiecare stare astfel:

1. **alege** o stare s arbitrară
2. for fiecare $s_j \in$ *calculeaza_succesori*(s)
3. $f(s_j) = c(s, s_j) + h[s_j]$
4. $h[s] \leftarrow min_{s_j} f(s_j)$

Căutare on-line : ADP

- pentru costul pozitive, obținem convergență
- dar pentru spații de stări mari, este nepractic
- ADP este baza pentru Real-time A*

Căutare on-line : LRTA*

- folosesc o tabelă cu valorile estimate $H[s]$ pentru fiecare stare
- actualizăm valorile pe măsură ce explorăm
- trebuie să considerăm costul pentru orice revenire
- revenim doar dacă costul estimat pe calea curentă depășește costul implicat de revenire

Căutare on-line : LRTA*

LRTA*(s) – întoarce acțiunea de realizat din s

global: H, $Rezultat$, s_{prev}, a_{prev} inițial, $s_{prev} \leftarrow \perp$

1. **dacă** s este stare finală **atunci întoarce** SUCCES
2. **dacă** s este stare nouă **atunci** $H[s] \leftarrow h(s)$ cu h funcția euristică
3. **dacă** $s_{prev} \neq \perp$ **atunci**
4. $Rezultat[s_{prev}, a_{prev}] \leftarrow s$
5. $H[s_{prev}] \leftarrow min_{b \in actiuni(s_{prev})} \begin{cases} c(s_{prev}, s') + H[s'] & \text{dacă există } s' = Rezultat[s_{prev}, b] \\ h(s_{prev}) & \text{, altfel} \end{cases}$
6. $s_{prev} \leftarrow s$
7. **întoarce** $b \in actiuni(s)$,

care minimizează $\begin{cases} c(s, s'') + H[s''] & \text{dacă există } s'' = Rezultat[s, b] \\ h(s) & \text{, altfel} \end{cases}$

Căutare on-line : LRTA*

- dacă
  - toate costurile sunt pozitive
  - există o cale de la orice stare la starea finală
  - funcția h este admisibilă

  atunci LRTA* este complet

- LRTA* se apropie de soluția optimă la parcurgeri succesive

Mulțumesc!

https://forms.gle/DJUdkejstkvyNRF5A

Feedbackul este binevenit!